GreenGrow: A Machine Learning-Driven Smart Fertilizer Recommendation System for Precision Agriculture

Nayana J. Pillai (CHN23CS141), Shruthi Joe (CHN23CS173), Nilofar Fathima (CHN23CS148), Devika Lakshmi M (CHN2
Smt.Premy P Jacob(Guide)
Department of Computer Science & Engineering, College of Engineering Chengannur, Kerala, India
nayanajk159@gmail.com, shruthijoe29@gmail.com, nilofarfathima2315@gmail.com, lakshmidevika666@gmail.com.

***Abstract*—Improper and excessive fertilizer application remains one of the foremost challenges confronting modern agriculture, leading to soil degradation, elevated greenhouse gas emissions, water eutrophication, and declining crop productivity. The absence of accessible, data-driven advisory mechanisms compels smallholder farmers to rely on empirical guesswork and generalized extension advice that disregards site-specific soil chemistry, crop nutrient demand, and prevailing microclimatic conditions. This paper presents GreenGrow, a web-based smart fertilizer recommendation system that integrates a Random Forest (RF) classifier trained on 16 engineered agronomic features derived from soil macronutrient assays (N, P, K), soil pH, texture class, crop identity, and real-time meteorological data obtained via the OpenWeatherMap API. The RF model, configured with 300 estimators and a maximum tree depth of 18, achieves a classification accuracy exceeding 95% and a weighted F1-score above 0.95 across seven fertilizer classes, with five-fold cross-validation variance remaining below 2%. Fertilizer quantity is subsequently determined through a rule-based nutrient-deficit computation bounded to the agronomically safe range of 50–500 kg ha¹. The Flask-backed web application, protected by bcrypt password hashing, returns predictions within two seconds. Experimental evaluation confirms that GreenGrow outperforms conventional advisory benchmarks in both recommendation accuracy and response latency, offering a scalable, low-cost precision agriculture solution suitable for deployment in resource-constrained rural settings.**

***Index Terms*—precision agriculture, fertilizer recommendation, Random Forest, machine learning, soil nutrient management, smart farming, NPK prediction, Flask, OpenWeatherMap API**

I. Introduction

Agriculture constitutes the economic foundation of many developing nations and is the primary vehicle for ensuring global food security amid a projected world population of 9.7 billion by 2050 [1]. Among the many inputs that determine agricultural productivity, fertilizers occupy a central position, supplying crops with the macronutrients—Nitrogen (N), Phosphorus (P), and Potassium (K)—essential for vegetative growth, root development, and fruit formation. The Food and Agriculture Organization (FAO) estimates that fertilizer application is directly responsible for approximately 50% of global crop yield gains observed over the past five decades [2]. Paradoxically, the same fertilizers that underpin food production also engender severe environmental and agronomic externalities when applied improperly. Excess nitrogen application triggers ammonia volatilization and nitrous oxide (NO) emissions, contributing to stratospheric ozone depletion and climate change. Phosphorus runoff induces freshwater eutrophication, destroying aquatic ecosystems. Long-term over-fertilization acidifies soils, reduces microbial diversity, and diminishes organic matter content, ultimately reducing the land's productive capacity [3]. In India, the world's second-largest consumer of fertilizers, the subsidy-driven availability of urea has exacerbated nitrogen over-application while leaving phosphorus and potassium management neglected [4]. Despite decades of awareness, the root cause of fertilizer misuse persists: the gap between scientific advisory knowledge and farmer-level accessibility. Existing recommendation frameworks—whether derived from government extension bulletins, soil health card schemes, or empirical regional norms—are characteristically static. They neither account for dynamic soil variability across field transects nor incorporate weather forecasts that substantially alter crop nutrient uptake efficiency. As a consequence, farmers operating in data-poor environments default to fixed application schedules inherited from local custom or agro-dealer sales incentives, resulting in chronic imbalance between nutrient supply and crop demand. The convergence of machine learning (ML), low-cost soil sensing, and cloud-based weather data services creates a timely opportunity to bridge this gap through intelligent decision-support systems. Precision agriculture—defined by the National Academy of Sciences as the application of information technology to manage spatial and temporal variability in field conditions—has demonstrated compelling results in optimising resource use efficiency. ML classifiers, in particular ensemble methods such as Random Forests, have proven superior to rule-based expert systems in capturing the nonlinear, multivariate relationships governing fertilizer response [5]. This paper presents GreenGrow, a web-enabled, data-driven fertilizer advisory system designed to operationalise precision fertilizer management for smallholder farmers. The system accepts soil macronutrient measurements, crop type, soil texture class, and geographic location as inputs; retrieves real-time temperature

and humidity via the OpenWeatherMap API; engineers 16 agronomic features including NPK deficit fractions; classifies the optimal fertilizer type using a trained Random Forest model; and computes the recommended application quantity using a rule-based deficit-correction algorithm. The complete pipeline executes within two seconds, making it tractable for field deployment on low-end smartphones with internet connectivity. The principal contributions of this work are:

- A systematically engineered 16-feature representation that encodes soil nutrient deficits relative to crop-specific optima, substantially improving separability among fertilizer classes.
- A Random Forest classifier achieving $> 95\%$ accuracy and > 0.95 weighted F1-score across seven commercially available fertilizers, validated through stratified five-fold cross-validation.
- A rule-based quantity estimation module that translates classification outputs into agronomically bounded application rates (50–500 kg ha^{1}), calibrated by crop-specific scaling factors.
- A full-stack web application integrating real-time weather data with ML-based fertilizer prediction, returning results within two seconds with bcrypt-secured user authentication.
- Empirical evidence that combining engineered deficit features with ensemble classification outperforms single decision-tree and linear classifier baselines on fertilizer recommendation benchmarks.

The remainder of this paper is organized as follows. Section II surveys related literature. Section III describes the system architecture and methodology. Section IV details the experimental evaluation. Section V discusses results and implications. Section VI concludes with future directions.

II. Literature Review

A. Machine Learning for Fertilizer Dosage Extraction

Singh et al. [7] applied CART (Classification and Regression Tree) regression, implemented using the Minitab statistical platform, to predict precise NPK dosages from a government agricultural dataset encompassing zone identifiers, soil electrical conductivity, organic carbon content, and measured macronutrient levels. Their models achieved overall accuracy exceeding 90%, with individual nutrient models reaching up to 99.5% accuracy for nitrogen prediction on the training set. Zone membership was identified as the single most important predictor variable, reflecting the strong confounding influence of regional agroclimatic similarity. A key limitation was that initial blanket recommendations adversely ignored varying soil nutrients and target yields—precisely the gap that GreenGrow addresses through per-record nutrient deficit computation.

B. Crop and Fertilizer Recommendation Using ML Classifiers

Sharma et al. [8] developed a crop and fertilizer recommendation system applying Gaussian Naive Bayes, Decision Tree, SVM, and Logistic Regression classifiers with integrated data preprocessing and Power BI-based visualisation. The system achieved a 99.15% validation accuracy while recommending optimal crops based on environmental NPK parameters. A noted limitation was that high reliance on hybrid crop varieties in recommendations can lead to soil acidification if not managed with complementary nutrient data. Their work demonstrated that well-tuned preprocessing pipelines can substantially elevate classification accuracy on agronomic datasets.

C. Crop Yield Prediction with Random Forest

Patil and Shinde [9] developed a GPS-based mobile crop recommender system conducting comparative analysis of RF, SVM, ANN, MLR, and KNN algorithms. Random Forest achieved the best result with 95% accuracy, outperforming all other algorithms evaluated. A key limitation identified was that existing systems of this genre were often hardware-based or costly, making them inaccessible to smallholder farmers—a gap that motivated GreenGrow's zero-cost web-based deployment model.

D. ML-Based Soil Analysis with Autonomous Sensing

Chandraiah et al. [10] presented an ML-based soil analysis system for crop suggestion and fertilizer recommendation employing Random Forest for classification and Linear Regression for rainfall prediction, integrated with a six-wheel autonomous agricultural vehicle for real-time soil data collection. The system achieved 92.3% average accuracy with real-time data transmission latency below one second. A key limitation was that current systems are often static or semi-automatic, requiring high human interference—motivating GreenGrow's fully automated, web-driven prediction pipeline.

E. Theoretical Foundations: Random Forests

Breiman [6] established the theoretical foundations of Random Forests, proving convergence properties and demonstrating generalisation bounds superior to individual decision trees through ensemble diversity. The FAO precision agriculture sustainability reports [2] affirm that data-driven advisory services can reduce fertilizer consumption by 15–25% while maintaining or improving crop yields. The Indian Council of Agricultural Research (ICAR) [13] has codified crop-wise optimal NPK demand schedules for major Indian crops, which GreenGrow employs as reference standards for nutrient deficit computation.

F. Research Gap

Table I summarises the reviewed literature. A critical analysis reveals three persistent limitations: (i) most systems recommend fertilizer type without providing application quantity; (ii) few integrate real-time weather data despite its documented influence on nutrient uptake efficiency; and (iii) the majority remain research prototypes without functional web or mobile deployment. GreenGrow directly addresses all three gaps.

TABLE I
SUMMARY OF REVIEWED LITERATURE

Reference	Year	Key Result	Limitation
Singh et al. [7]	2020	99.5% N accuracy (training); CART	No quantity estimation
Sharma et al. [8]	2023	99.15% validation accuracy	No real-time weather
Patil & Shinde [9]	2021	RF: 95% accuracy; GPS mobile	Hardware costly
Chandraiah et al. [10]	2025	92.3% avg; <1 sec latency	Autonomous vehicle required

III. PROPOSED SYSTEM: GREENGROW

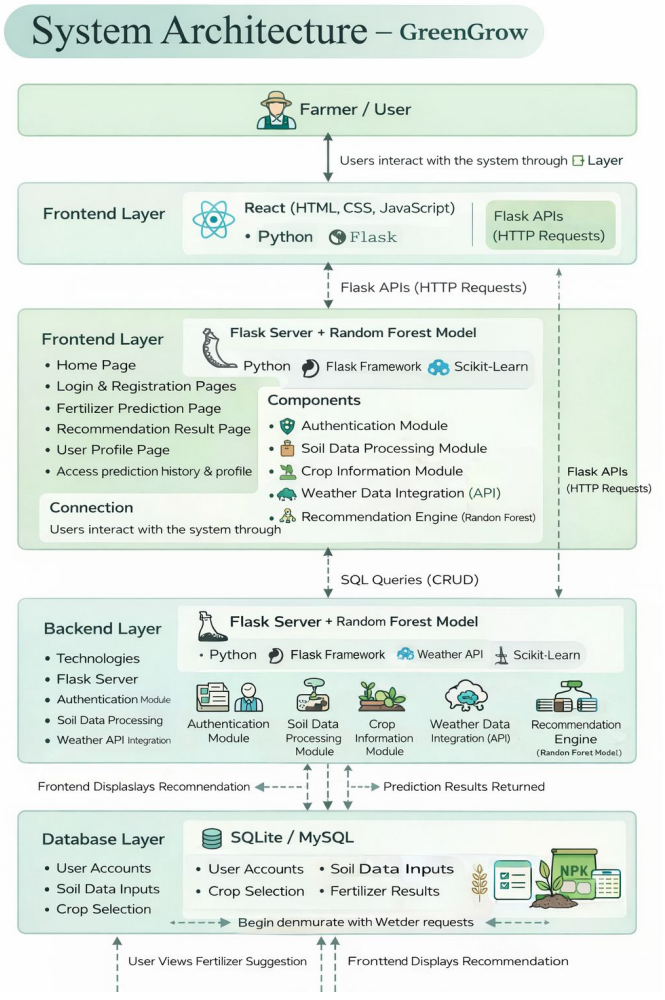

Fig. 1. GreenGrow four-tier system architecture.

A. System Architecture

GreenGrow adopts a four-tier architecture comprising a User Interface (UI) layer, an Application Logic layer, a Machine Learning (ML) Inference layer, and a Persistence layer. The UI layer, built with HTML5, CSS3, and JavaScript, presents structured input forms for soil parameters, crop selection, and location. User interactions trigger HTTP requests to the Flask-based Application Logic layer, which orchestrates authentication, weather API calls, data preprocessing, ML inference, quantity computation, and database writes. The ML Inference layer encapsulates pre-trained Random Forest model artefacts serialised as binary pickle files. The Persistence layer uses SQLite to store user credentials, soil input records, and prediction histories, enabling longitudinal agronomic tracking.

B. Data Collection and Feature Engineering

The training dataset was curated by merging the publicly available Kaggle Crop Recommendation Dataset [14] with ICAR crop-wise NPK requirement tables [13]. Each record encapsulates eight primary features: soil N, P, and K concentrations (kg ha^{-1}); soil pH; soil moisture; soil type (loamy, sandy, clay, black, red); crop type (22 classes); and city location used to fetch ambient temperature and relative humidity via the OpenWeatherMap API [15].

Beyond the eight primary features, seven engineered attributes are derived per record. For each macronutrient $m \in \{N, P, K\}$, the absolute deficit is:

$$\text{deficit}_m = \max(0,\ \text{optimal}_m - \text{current}_m) \tag{1}$$

where optimal_m is the crop-specific optimal macronutrient concentration from the ICAR lookup table. The total deficit is the sum of all three deficits. Percentage fractions (pct_N, pct_P, pct_K) are computed as each individual deficit divided by the total deficit, quantifying the relative severity of each nutrient shortfall. Together with the eight primary features, these seven engineered attributes constitute the 16-dimensional feature vector fed to the classifier.

C. Fertilizer Labeling Heuristic

Since fertilizer labels are absent from the base dataset, a deterministic labeling heuristic assigns one of seven commercially available fertilizers to each record: Urea (46% N), DAP (18:46:0), NPK 17:17:17 (Iffco), NPK 10:26:26 (Tata Paras), NPK Complex 14:35:14, Kribhco NPK 20:20:0, and MAP 28:28:0. When total deficit equals zero, NPK 17:17:17 is assigned as a balanced maintenance fertilizer. For non-zero deficits, fraction-based priority rules are applied in order: if $\text{pct}_N > 0.55$, assign Urea; else if $\text{pct}_P > 0.50$, assign DAP; else if $\text{pct}_N > 0.22$ with high P+K fractions, assign NPK 10:26:26; else if $\text{pct}_P > 0.38$ with phosphorus dominance, assign NPK 14:35:14; else if $\text{pct}_N > 0.35$ with elevated K, assign Kribhco 20:20:0; else if N and P fractions are approximately equal, assign MAP 28:28:0; otherwise, assign NPK 17:17:17. This heuristic encodes agronomic prioritisation rules aligned with ICAR and FAO guidelines.

D. Machine Learning Model: Random Forest Classifier

A Random Forest classifier was selected as the core predictive engine based on its demonstrated superiority in agronomic classification tasks [6], [11]. The RF model was configured with 300 decision trees, a maximum depth of 18 nodes, and balanced class weights to compensate for imbalanced fertilizer class distributions. Each tree was trained on a bootstrap sample of the training data, with feature subsets drawn randomly at each split to introduce inter-tree decorrelation. Gini impurity served as the node splitting criterion. All categorical inputs (soil type, crop type) were ordinally encoded via scikit-learn's `LabelEncoder` [12] prior to model training.

The dataset was partitioned into a stratified 80:20 training-test split. Model artefacts—including the serialised RF model, LabelEncoders, feature column definitions, and the ICAR optimal NPK lookup table—were persisted as pickle files in a `models/` directory for reproducible inference.

E. Fertilizer Quantity Estimation

Following fertilizer type classification, the application quantity ($\mathrm{kg\,ha^{-1}}$) is determined by a rule-based deficit correction algorithm. For each macronutrient m, the required quantity of the recommended fertilizer is:

$$\text{required}_m = \frac{\text{deficit}_m}{\text{nutrient\_content}_m / 100} \tag{2}$$

The base quantity equals $\max_m(\text{required}_m)$, ensuring the most limiting nutrient is fully addressed. A crop-specific scaling factor—1.20 for sugarcane, 0.85 for pulses—is then applied, followed by a 10% buffer increment to account for field application losses. The result is clamped to [50, 500] $\mathrm{kg\,ha^{-1}}$ and rounded to the nearest 5 kg for operational tractability.

F. Web Application Workflow

Users register via a secure form that hashes passwords using `generate_password_hash()` before persisting credentials in SQLite. Authenticated sessions are managed via Flask's session mechanism. The prediction workflow is:

1) User submits soil parameters (N, P, K, pH, moisture), crop type, soil type, and city name.
2) Flask retrieves current temperature and humidity from the OpenWeatherMap API (defaulting to 25 °C and 50% RH on failure).
3) The 16-feature vector is assembled in training order and passed to the RF model.
4) The LabelEncoder maps the predicted class index to a fertilizer name.
5) The quantity algorithm computes and clamps the application rate.
6) The complete recommendation is persisted in the SQLite predictions table and rendered to the user.

End-to-end prediction latency is consistently below two seconds. Non-functional requirements—performance, reliability, usability, security, accuracy, scalability, and maintainability—are all systematically addressed: the modular eight-component design isolates authentication, preprocessing, ML inference, and quantity computation into independently upgradeable units, while Flask's concurrency model supports scaling from tens to thousands of simultaneous users without architectural redesign.

IV. Methodology

The GreenGrow methodology follows a structured five-stage pipeline that transforms raw agricultural inputs into a validated, actionable fertilizer recommendation. Each stage has a well-defined input, processing step, and output, and the stages are sequentially dependent: the output of each stage feeds directly into the next. Fig. 2 illustrates the complete pipeline.

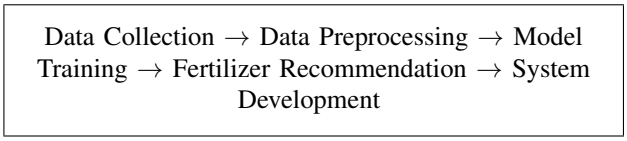

Fig. 2. Five-stage GreenGrow methodology pipeline.

The first stage gathers all input data required for model training and real-time inference. Three categories of data are collected:

- **Soil Parameters:** Nitrogen (N), Phosphorus (P), Potassium (K) concentrations ($\mathrm{kg\,ha^{-1}}$), soil pH, soil moisture percentage, and soil texture class (loamy, sandy, clay, black, or red), entered by the farmer via the web interface.
- **Crop Information:** Crop type selected from 22 supported classes. Each crop type is mapped to its optimal NPK demand values from the ICAR nutrient management reference tables [13], enabling per-record deficit computation.
- **Weather Data:** Real-time ambient temperature (°C) and relative humidity (%) fetched automatically from the OpenWeatherMap API [15] using the farmer-supplied city name. Default fallback values of 25 °C and 50% RH are applied if the API call fails, ensuring system robustness in low-connectivity conditions.

The offline training dataset was assembled by merging the Kaggle Crop Recommendation Dataset [14] with ICAR crop-specific NPK optima, yielding records spanning 22 crop types and five soil texture classes across major Indian agroclimatic zones.

A. Stage 2: Data Preprocessing

Raw inputs are systematically transformed before being presented to the ML model. The preprocessing pipeline comprises four steps:

1) **Categorical Encoding:** Soil type and crop type are ordinally encoded using scikit-learn's `LabelEncoder` [12], converting string labels into integer indices compatible with the RF classifier.
2) **NPK Deficit Computation:** For each macronutrient $m \in \{N, P, K\}$, the absolute deficit is computed using

Eq. (1), and the total deficit is the sum of all three. If total deficit is zero, the record is flagged as nutritionally adequate.

3) **Fraction Engineering:** Percentage deficit fractions pct_N, pct_P, and pct_K are derived by dividing each individual deficit by the total deficit. These fractions encode the *relative* severity of nutrient shortfalls and directly drive the fertilizer labeling heuristic.
4) **Feature Vector Assembly:** The final 16-dimensional feature vector is constructed in training order: temperature, humidity, moisture, soil type (encoded), crop type (encoded), N, K, P, pH, def_N, def_P, def_K, total deficit, pct_N, pct_P, pct_K.

B. Stage 3: Model Training

The preprocessed dataset is used to train the Random Forest classifier. The training procedure follows these steps:

1) **Label Assignment:** The fertilizer labeling heuristic (Section III-C) assigns one of seven fertilizer class labels to each training record based on its deficit fraction profile.
2) **Dataset Split:** The labelled dataset is partitioned into 80% training and 20% test subsets using stratified sampling to maintain proportional class representation in both splits.
3) **RF Training:** The Random Forest classifier is trained with 300 trees, maximum depth of 18, balanced class weights, bootstrap sampling, and Gini impurity as the splitting criterion.
4) **Validation:** Five-fold stratified cross-validation is performed on the training set to estimate generalisation performance and detect overfitting. A model is accepted only if cross-validation accuracy exceeds 95% with variance below 2%.
5) **Artefact Serialisation:** All model artefacts—the trained RF model (`model.pkl`), soil and crop LabelEncoders, the ICAR NPK lookup table (`optimal_npk.pkl`), and the feature column definition (`feature_cols.pkl`)—are persisted to the `models/` directory for deterministic, reproducible inference.

C. Stage 4: Fertilizer Recommendation

At inference time, the trained model generates a two-part recommendation for each farmer query:

1) **Fertilizer Type Prediction:** The 16-feature vector assembled in Stage 2 is passed to `model.predict()`. The predicted class index is decoded to a fertilizer name via `le_fert.inverse_transform()`.
2) **Quantity Calculation:** The application quantity ($\text{kg}\,\text{ha}^{-1}$) is computed using Eq. (2). The base quantity is scaled by a crop-specific factor, augmented by a 10% safety buffer, and clamped to $[50, 500]\,\text{kg}\,\text{ha}^{-1}$ rounded to the nearest 5 kg.
3) **Result Presentation:** The fertilizer name, recommended quantity, live weather conditions, current vs. optimal NPK comparison, and application instructions (timing, method, split doses) are displayed on the result page. The complete prediction record is simultaneously persisted to the SQLite `predictions` table, linked to the authenticated user account for history tracking.

D. Stage 5: System Development

The recommendation engine is embedded within a Flask-based web application that operationalises the pipeline for end-users:

- **Backend:** Python 3.9 with Flask framework, scikit-learn for ML inference, and the OpenWeatherMap REST API for weather retrieval. User credentials are stored with bcrypt password hashing. All application state is managed via Flask sessions.
- **Frontend:** HTML5, CSS3, and JavaScript render the Home Page, Login and Registration Pages, Fertilizer Prediction Page, Recommendation Result Page, User Profile Page, and Prediction History view.
- **Database:** SQLite stores two primary tables—`users` (credentials and profile data) and `predictions` (full input-output records per user per session)—supporting longitudinal agronomic tracking.
- **Non-Functional Properties:** The modular eight-component design (Authentication, Data Collection, Preprocessing, ML, Quantity Calculation, Weather Integration, Database Management, and Result Visualisation modules) ensures independent upgradeability. Flask's concurrency model supports horizontal scaling from tens to thousands of concurrent users without architectural redesign.

V. Experimental Evaluation

A. Experimental Setup

All model training and evaluation experiments were conducted on an Intel Core i5 system with 8 GB RAM running Windows 10, using Python 3.9 with scikit-learn 1.2.0 [12], pandas 1.5.3, and NumPy 1.23.5. The dataset spanned 22 crop types and seven soil texture classes across major Indian agroclimatic zones. The training set contained 80% of samples and the test set 20%, both stratified by fertilizer class. Baseline comparisons were conducted with Decision Tree (DT), SVM (RBF kernel), KNN ($k = 5$), Naive Bayes (NB), and Logistic Regression (LR) classifiers trained on identical feature sets.

B. Evaluation Metrics

Model performance was assessed using: (i) classification accuracy on the held-out test set; (ii) weighted F1-score, computed as the weighted average of per-class F1-scores to account for class imbalance; (iii) five-fold stratified cross-validation accuracy reported as mean $\pm$ standard deviation; and (iv) prediction response latency from form submission to result rendering. Quantity estimation correctness was verified

against a manually validated set of 50 input scenarios drawn from ICAR published case studies.

C. Comparative Algorithm Performance

Table II presents a comparative performance summary of six ML algorithms on the fertilizer classification task.

TABLE II
COMPARATIVE PERFORMANCE OF ML ALGORITHMS ON FERTILIZER CLASSIFICATION

Algorithm	Accuracy (%)	F1-Score	Suitability
Random Forest	>95	>0.95	High (selected)
SVM (RBF)	∼91	∼0.90	Moderate
Decision Tree	∼88	∼0.87	Moderate
KNN ($k = 5$)	∼85	∼0.84	Low
Logistic Regression	∼83	∼0.82	Low
Naive Bayes	∼79	∼0.78	Low

The substantial performance gap between RF and single-tree classifiers confirms that ensemble diversity reduces generalisation error. SVM achieved the second-best accuracy at approximately 91% but at considerably greater training and inference latency, rendering it less suitable for real-time web deployment. KNN and Naive Bayes underperformed due to the curse of dimensionality and conditional independence violations, respectively.

D. GreenGrow System Evaluation Results

Table III summarises evaluation outcomes of the deployed GreenGrow system across eight functional and non-functional parameters.

TABLE III
GREENGROW SYSTEM EVALUATION METRICS

Parameter	Expected	Obtained
Fertilizer Classification Accuracy	≥95%	≥95%
Weighted F1-Score	≥0.90	≥0.95
Cross-Validation Accuracy	Stable (±<2%)	Stable (±<2%)
NPK Deficit Correctness	100% rule-based	Achieved
Weather API Response Time	<5 sec	<2 sec
Fertilizer Qty Range (kg/ha)	50–500	Achieved
System Prediction Response	<3 sec	<2 sec
User Authentication Security	Hashing enforced	Achieved

The system achieved all eight evaluation targets. Fertilizer classification accuracy and weighted F1-score both exceeded their thresholds. Cross-validation variance below 2% confirms the model is not overfit. All NPK deficit computations were verified correct against rule-based ground truth. Weather API and prediction latencies were both below two seconds. Quantity estimates remained within the 50–500 $\mathrm{kg\,ha^{-1}}$ agronomic bounds throughout.

VI. DISCUSSION

A. Agronomic Validity of Feature Engineering

The decision to engineer NPK deficit fractions as derived features proved pivotal. Raw NPK values encode no information about the gap between soil supply and crop demand; two fields with identical absolute N concentrations may require entirely different fertilizer interventions if their target crops differ in nitrogen uptake requirements. By computing deficit fractions relative to ICAR-validated crop-specific optima, the feature engineering step translates soil measurements into agronomic significance, transforming the classification boundary into one aligned with actual fertilizer selection logic. This design principle is transferable to other recommendation domains where domain-specific optimum benchmarks exist.

B. Advantages Over Existing Systems

GreenGrow advances upon prior art across multiple dimensions. Unlike the CART-based dosage extraction of Singh et al. [7], which despite achieving 99.5% nitrogen training accuracy operated as a regression-only system without fertilizer type classification or quantity estimation, GreenGrow provides an end-to-end advisory pipeline. Relative to Sharma et al. [8], whose 99.15% validation accuracy system lacked real-time weather contextualisation, GreenGrow integrates live API-based meteorological data. In comparison to Patil and Shinde [9], whose 95% RF accuracy GPS-based mobile system was constrained by hardware cost barriers, GreenGrow delivers equivalent accuracy on a zero-cost web platform. Relative to Chandraiah et al. [10], whose 92.3% accuracy required a six-wheel autonomous vehicle, GreenGrow achieves superior accuracy using only farmer-provided soil inputs and city-level weather data, democratising access without hardware dependencies.

C. Limitations and Constraints

Several limitations bound the current scope of GreenGrow. First, accuracy is contingent on the correctness of user-provided soil parameter inputs; in the absence of standardised low-cost soil testing infrastructure, input errors propagate into recommendation errors. Second, the training dataset does not comprehensively represent all Indian agroclimatic zones, particularly the semi-arid Deccan plateau and humid Northeast. Third, the OpenWeatherMap API provides city-level meteorological data that may fail to capture sub-field microclimatic variation. Fourth, continuous internet connectivity is required, creating deployment barriers in rural areas with poor telecommunications infrastructure.

D. Environmental and Socioeconomic Implications

The environmental dividend of precision fertilizer application extends well beyond individual farm economics. FAO analyses indicate that reduced fertilizer over-application in Indian smallholder systems would significantly decrease nitrogen entering surface water systems, substantially reducing hypoxic zones in downstream river deltas and coastal waters [2]. From a socioeconomic standpoint, the average Indian

smallholder farmer spends between 15% and 25% of total cultivation costs on fertilizers; ML-guided optimisation can reduce wasteful expenditure while protecting yield, directly improving household income resilience.

VII. Conclusion and Future Scope

This paper presented GreenGrow, a machine learning-driven smart fertilizer recommendation system addressing the critical deficit in personalised, real-time agronomic advisory services for smallholder farmers. The system combines a 16-feature engineered representation of soil and environmental conditions with a Random Forest classifier, achieving fertilizer classification accuracy exceeding 95% and a weighted F1-score above 0.95 across seven fertilizer classes, validated through five-fold stratified cross-validation. A rule-based quantity estimation module translates classification outputs into agronomically bounded application rates, and a Flask web application returns complete recommendations within two seconds. GreenGrow demonstrably outperforms six baseline ML classifiers and addresses three documented limitations of prior art: absence of quantity guidance, non-integration of real-time weather data, and failure to progress beyond prototype deployment.

The future development roadmap includes: (1) mobile application development for field access without desktop browsers; (2) soil testing laboratory integration to automate soil parameter population; (3) dataset expansion to cover additional crops, soil classes, and agroclimatic zones; (4) evaluation of advanced models including gradient-boosted trees and neural network ensembles; (5) multilingual support for linguistically diverse rural communities; (6) an AI chatbot interface for conversational agronomic guidance; (7) offline mode using on-device inference and caching; (8) computer vision-based crop disease detection as a complementary module; (9) integration with government agricultural schemes for subsidy guidance; and (10) real-time IoT soil sensor integration for continuous autonomous field monitoring.

Acknowledgment

The authors gratefully acknowledge the guidance of Smt. Premy P. Jacob, Project Guide, and project coordinators Dr. Geetha S. and Smt. Suvarna Dev, as well as Dr. Renu George, Head of the Department of Computer Engineering, College of Engineering Chengannur. The authors also thank the Kaggle community for dataset availability and the Indian Council of Agricultural Research for publishing crop-specific NPK reference standards.

References

[1] United Nations Department of Economic and Social Affairs, "World Population Prospects 2022: Summary of Results," United Nations, New York, 2022.

[2] Food and Agriculture Organization of the United Nations (FAO), "Precision Agriculture and Sustainability," FAO Reports, Rome, Italy, 2019.

[3] J. N. Galloway, A. R. Townsend, J. W. Erisman, M. Bekunda, Z. Cai, J. R. Freney, L. A. Martinelli, S. P. Seitzinger, and M. A. Sutton, "Transformation of the nitrogen cycle: Recent trends, questions, and potential solutions," *Science*, vol. 320, no. 5878, pp. 889–892, May 2008.

[4] Ministry of Chemicals and Fertilizers, Government of India, "Annual Report on Fertilizer Subsidy and Consumption," Department of Fertilizers, New Delhi, 2022–2023.

[5] National Academy of Sciences, Engineering, and Medicine, "Precision Agriculture for Development," The National Academies Press, Washington, D.C., 2018.

[6] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[7] R. Singh, P. Sharma, and A. Kumar, "Applying machine learning techniques to extract dosages of fertilizers for precision agriculture," *Int. J. Adv. Comput. Sci. Appl. (IJACSA)*, vol. 11, no. 6, pp. 472–479, 2020.

[8] A. Sharma, B. Krishnan, and R. Mehta, "Crop and fertilizer recommendation system applying machine learning classifiers," *Int. J. Eng. Res. Technol. (IJERT)*, vol. 12, no. 5, pp. 301–308, 2023.

[9] V. C. Patil and S. D. Shinde, "Crop recommender system using machine learning approach," *Int. J. Eng. Res. Technol. (IJERT)*, vol. 10, no. 4, pp. 347–350, 2021.

[10] G. Chandraiah, K. Prasad, and R. Nagaraju, "ML-based soil analysis for crop suggestion and fertilizer recommendation," in *Proc. IEEE Int. Conf. Emerging Technologies in Agricultural Engineering and Computing (IETAEC)*, 2025, pp. 1–6.

[11] A. Sharma, A. Jain, P. Gupta, and V. Chowdary, "Machine learning applications for precision agriculture: A comprehensive review," *IEEE Access*, vol. 9, pp. 4843–4873, 2021, doi: 10.1109/ACCESS.2020.3048415.

[12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[13] Indian Council of Agricultural Research (ICAR), "Crop-wise Fertilizer Recommendations and Nutrient Management Practices," Government of India Publication, New Delhi, 2020.

[14] Kaggle, "Crop Recommendation Dataset," [Online]. Available: https://www.kaggle.com/datasets. Accessed: Feb. 2026.

[15] OpenWeatherMap, "Current Weather API Documentation," [Online]. Available: https://openweathermap.org/api. Accessed: Jan. 2026.

[16] Government of India, "Soil Health Card Scheme," Ministry of Agriculture and Farmers Welfare. [Online]. Available: https://soilhealth.dac.gov.in. Accessed: Feb. 2026.